

Preparing markdown files for Dig Deeper

Every `.md` file you drop becomes a Dig Deeper entry. This is how to get one ready, from NotebookLM download to the `raw/` folder.

Roughly one minute per file.

Where things go

```
D:\projects\logos-atlas-content\  
└─ ep01_New Heaven and New Earth\  
    └─ raw\  
        │ └─ en\          ← you work here  
        │ └─ id\  
        └─ ready\        ← Build-Ready.ps1 writes here. Never edit by hand.
```

Always edit in `raw\`. `Build-Ready.ps1` copies markdown from raw to ready on every run — anything you add in `ready\` gets overwritten the next time you build.

Step 1 — Download from NotebookLM

Open the Studio output you want — briefing doc, study guide, mindmap, FAQ, timeline, whatever it is.

Export as Markdown. If only Google Docs is offered, open it there and use **File → Download → Markdown (.md)**.

Save into `raw\en\`. Don't worry about the name yet.

Step 2 — Rename it

```
epNN_<short-name>_<lang>.md
```

The name is now just an identifier — the title readers see comes from inside the file, so this can be short.

```
ep01_kainos_en.md  
ep01_mindmap_en.md  
ep01_briefing_en.md  
ep01_full-script_en.md
```

Rules: two-digit episode number, lowercase, dashes inside the name, underscores only between the three parts, language code always present including `_en`.

Step 3 — Open it and add front-matter

Open in Notepad, VS Code, or any plain-text editor. Put these five lines at the very top — before everything, no blank line above:

```
markdown

---
title: Kainos or Neos — why the Greek word matters
description: The linguistic argument behind renewal versus replacement, and why
---
```

Then a blank line, then the existing content.

Writing these two fields:

`title` is what a reader scans in the dropdown. Write it as a heading, not a label. "Kainos or Neos — why the Greek word matters", not "Kainos analysis".

`description` is one sentence answering *why would I open this?* Keep it under about 100 characters — it renders as a second line under the title on a phone.

Weak

title: Briefing

description: A briefing document

Better

title: The four passages that promise a new earth

description: Every place scripture makes the promise, and what each one adds.

Step 4 — Save as UTF-8

This one matters. If you save as ANSI, apostrophes and quotation marks arrive as `â€™` and `â€œ` on the published page.

In Notepad: File → Save As → **Encoding** dropdown at the bottom → **UTF-8** (not "UTF-8 with BOM"). Save over the same file.

In VS Code: the encoding shows in the bottom-right status bar. It should say `UTF-8`. If it says anything else, click it → Save with Encoding → UTF-8.

Step 5 — Check it

```
powershell

cd "D:\projects\logos-atlas-content\ep01_New Heaven and New Earth\raw\en"
Get-Content ep01_kainos_en.md -TotalCount 5
```

You want to see:

```
---  
title: Kainos or Neos – why the Greek word matters  
description: The linguistic argument behind renewal versus replacement.  
---
```

If line 1 isn't exactly `---`, the front-matter won't parse and the app falls back to the filename.

Check the whole file for encoding damage:

```
powershell  
  
Select-String -Path *.md -Pattern "â€|Ã©|â€™"
```

No output is what you want. Any match means a file was saved in the wrong encoding — repair it:

```
powershell  
  
$p = "D:\projects\logos-atlas-content\ep01_New Heaven and New Earth\raw\en\ep01  
$t = [System.IO.File]::ReadAllText($p, [System.Text.Encoding]::GetEncoding(1252  
[System.IO.File]::WriteAllText($p, $t, (New-Object System.Text.UTF8Encoding($fa
```

Step 6 — Repeat, then build

Do steps 1-5 for each Dig Deeper file. Then:

```
powershell  
  
cd D:\projects\logos-atlas-content  
.\Build-Ready.ps1 -EpisodePath ".\ep01_New Heaven and New Earth" -EpisodeNumber
```

Every `.md` in `raw\en\` lands in `ready\` with its front-matter intact.

Step 7 — Publish

Drag everything from `ready\` into the drop zone. Each markdown file appears as its own Dig Deeper row, showing the parsed title and description.

Read those before pressing Publish. That staging row is exactly what readers will see. If a title reads badly, fix the file and re-drop — much cheaper than after publishing.

Other languages

Same process in `raw\id\`, and **the front-matter translates too**:

markdown

```
---  
title: Kainos atau Neos – mengapa kata Yunani ini penting  
description: Argumen linguistik di balik pembaruan versus penggantian.  
---
```

That's the point of putting it inside the file — each language carries its own title and description, and the app picks up the right one automatically.

Order of entries follows the order files are read, so keep the same names across languages if you want the same order.

Quick reference

Edit in	<code>raw\<lang>\</code> — never <code>ready\</code>
Name	<code>epNN_short-name_lang.md</code>
First line	<code>---</code> exactly, nothing above it
Fields	<code>title</code> and <code>description</code>
Encoding	UTF-8, no BOM
Check	<code>Get-Content <file> -TotalCount 5</code>
Then	<code>Build-Ready.ps1</code> , then drag <code>ready\</code> into the drop zone

One habit worth keeping: write the front-matter the moment you rename the file. Both edits happen in the same breath, and a file that reaches `ready\` without front-matter falls back to its filename — which still works, but reads like a filename.